

# Entrega Final - Segundo Parcial

*Compilado automático de documentos y evidencia*

---

## README

### bigdata - Cloud Provider Analytics

Este proyecto fue realizado por Jeremias Feferovich, Felipe Mindlin, Martin Zahnd, Florencia Carrica y Gianfranco Magliotti, en el contexto de la materia 72.80 - Big Data.

#### Contexto

El proyecto simula el rol del equipo de datos de un proveedor cloud que debe cubrir: - metricas operativas near real-time de uso/costo - procesamiento batch diario/mensual para maestros y facturacion

El pipeline esta pensado para datos con nullos, duplicados, inconsistencias y evolucion de schema (v1/v2).

#### Estado actual

- Primer parcial: disenio preliminar completo en docs/primer\_parcial\_diseno\_preliminar.md.
- Segundo parcial (MVP tecnico): implementado en PySpark + Cassandra scripts.

#### Estructura del repo

- src/pipeline/: jobs por capa (bronze\_batch.py, bronze\_stream.py, silver.py, gold.py, cassandra\_loader.py)
- scripts/run\_mvp.py: runner end-to-end del MVP tecnico
- cassandra/schema.cql: keyspace y tablas
- cassandra/queries\_minimas.cql: consultas minimas (#1 y #2)
- plan/roadmap\_entregas.md: roadmap actualizado (Parcial 2 + Final)
- docs/segundo\_parcial/mvp\_checklist.md: checklist de evidencia de entrega
- docs/segundo\_parcial/bitacora\_apropiacion\_tecnica.md: decisiones y trade-offs

#### Quickstart (MVP segundo parcial)

1. Instalar dependencias:

```
python -m pip install -r requirements.txt
```

2. Ejecutar pipeline local (sin carga Cassandra):

```
python scripts/run_mvp.py
```

3. Ejecutar pipeline con carga a Cassandra:

```
python scripts/run_mvp.py --with-cassandra --cassandra-host 127.0.0.1 --cassandra-port 9042
```

4. Crear schema y correr consultas:

```
cqlsh -f cassandra/schema.cql
```

```
cqlsh -f cassandra/queries_minimas.cql
```

## Nota

Los paths de landing esperados estan definidos en `src/pipeline/config.py` bajo `datalake/landing/`.

---

## Diseño preliminar (Primer Parcial)

# Cloud Provider Analytics

## Primer Parcial - Diseño preliminar

Fecha: 2026-04-16

Proyecto: Cloud Provider Analytics

Curso: Big Data (Primer Cuatrimestre 2026)

---

Documento de diseño preliminar orientado a validar viabilidad técnica temprana y dejar una base directa para la implementación del MVP técnico (segundo parcial) y la expansión de alcance en la entrega final.

## Resumen ejecutivo

- Arquitectura seleccionada: Lambda (batch + streaming).
- Almacenamiento intermedio: Parquet por zonas Landing/Bronze/Silver/Gold.
- Serving analítico: AstraDB/Cassandra (keyspace `cloud_analytics`) con modelado query-first.
- Objetivo del primer parcial: validar diseño y plan de ejecución, evitando sobre-ingeniería.

## Contexto

Como equipo de datos de un cloud provider, necesitamos cubrir dos necesidades en paralelo: - Near real-time para métricas operativas de uso/costo - Batch diario o mensual para maestros y facturación

Los datos llegan con nulos, inconsistencias, duplicados y evolución de schema (v1/v2), por lo que la arquitectura debe priorizar robustez e idempotencia sin sobre-complicar la primera fase.

## Objetivo de esta entrega

Presentar un diseño preliminar claro, visual y accionable para validar viabilidad técnica temprana. El alcance se mantiene conciso (evitando over-engineering), pero dejando base directa para el MVP técnico del segundo parcial y el cierre del final.

### 1) Diagrama de arquitectura de alto nivel

Patrón elegido: **Lambda** - Batch para maestros/facturación/encuestas. - Streaming para eventos de uso en near real-time. - Convergencia en Gold y Serving con vista unificada de negocio.

Justificación del patrón: - Evita forzar todo a stream (Kappa) cuando parte de los datos son naturalmente batch. - Reduce complejidad inicial en Colab, sin perder escalabilidad conceptual para etapas futuras.

## Vista de alto nivel

### Fuentes de datos

```
| - (batch)  customers_orgs, users, resources, support_tickets,
|           marketing_touches, nps_surveys, billing_monthly
| - (stream) usage_events_stream/*.jsonl
|
v
```

```
+-----+
| DATA LAKE (Parquet por zonas) |
|
| +-----+ +-----+ +-----+ +-----+ |
| | LANDING |-->| BRONZE  |-->| SILVER  |-->| GOLD   | |
| | Raw      |   | Batch +  |   | Conform + |   | Marts  | |
| | immutable|   | Streaming|   | Calidad  |   | negocio| |
| +-----+ +-----+ +-----+ +-----+ |
+-----+
```

```
|
v
+-----+ +-----+
| SERVING | | BI / Visualización|
| AstraDB/Cassandra |---->| Superset / Grafana|
| keyspace: cloud_analytics | | Tableau / PowerBI |
+-----+ +-----+
```

## Capa 1 - Landing

Fuentes de entrada (raw immutable)

- | - customers\_orgs.csv      (maestro de organizaciones)
- | - users.csv                (usuarios por organización)
- | - resources.csv            (VMs, contenedores, DBs, etc.)
- | - support\_tickets.csv      (tickets de soporte)
- | - marketing\_touches.csv    (interacciones de marketing)
- | - nps\_surveys.csv          (encuestas NPS)
- | - billing\_monthly.csv      (facturación mensual)
- | - usage\_events\_stream/    (eventos JSONL con schema v1 y v2)
  - | - part-0001.jsonl
  - | - part-0002.jsonl
  - | - ...

Salida: archivos crudos inmutables, sin transformación

Objetivo: mantener referencia fiel de origen para auditoría y reproceso.

## Capa 2 - Bronze

Landing

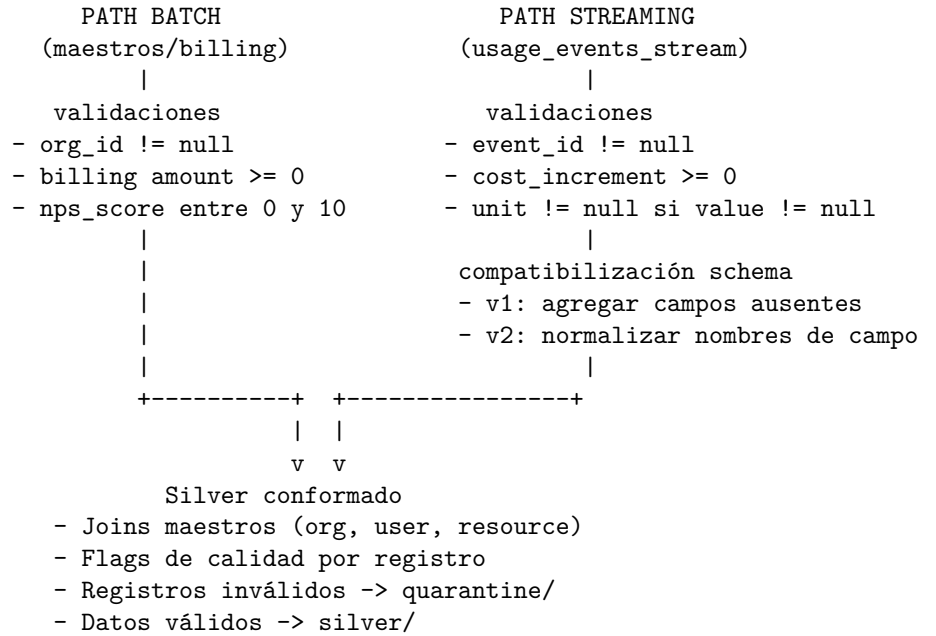
```
|--(Batch ingest: PySpark DataFrame)-----> Bronze Batch (Parquet)
|   maestros + facturación
|
|--(Structured Streaming ingest: PySpark)-----> Bronze Stream (Parquet)
|   usage_events_stream/*.jsonl
```

Controles aplicados en ambos paths:

- Schema explícito declarado (StructType)
- Campos de trazabilidad: ingest\_ts, source\_file
- [Stream] watermark: 1 hora sobre event\_ts
- [Stream] deduplicación por event\_id dentro de watermark
- [Stream] checkpointing en disco para idempotencia
- Particionado: fecha de ingesta (ingest\_date)

Objetivo: estandarizar estructura mínima y asegurar trazabilidad técnica temprana.

### Capa 3 - Silver



Objetivo: convertir datos crudos en datos analíticamente confiables.

## Capa 4 - Gold

Silver (batch + stream convergidos)

|                                     |                                               |
|-------------------------------------|-----------------------------------------------|
|                                     |                                               |
| +-- gold/org_daily_usage_by_service | (FinOps: uso diario por org y servicio)       |
|                                     |                                               |
| +-- gold/revenue_by_org_month       | (FinOps: revenue mensual por org)             |
|                                     |                                               |
| +-- gold/cost_anomaly_mart          | (FinOps: anomalías de costo por org/servicio) |
|                                     |                                               |
| +-- gold/tickets_by_org_date        | (Soporte: tickets por org y fecha)            |
|                                     |                                               |
| +-- gold/genai_tokens_by_org_date   | (Producto/GenAI: consumo de tokens por org)   |

Salida: datasets orientados a consultas de negocio, particionados por fecha

Objetivo: exponer métricas y KPIs listos para consumo analítico.



## Capa 5 - Serving

Gold -> AstraDB/Cassandra  
keyspace: cloud\_analytics

Tablas y modelo query-first:

| TABLA                      | PRIMARY KEY                            |
|----------------------------|----------------------------------------|
| org_daily_usage_by_service | ((org_id, service), usage_date DESC)   |
| revenue_by_org_month       | ((org_id), month DESC)                 |
| cost_anomaly_mart          | ((org_id, service), anomaly_date DESC) |
| tickets_by_org_date        | ((org_id), ticket_date DESC, severity) |
| genai_tokens_by_org_date   | ((org_id), usage_date DESC)            |

Escritura: foreachBatch (streaming) o Spark-Cassandra connector (batch)

Consultas: CQL (mínimo 2 en Parcial 2, 5 en Final)

Consumo final: Superset / Grafana / Tableau / PowerBI

Objetivo: responder consultas operativas y ejecutivas con baja latencia.

## 2) Mapeo de requisitos a componentes

| Requisito clave                              | Componente / tecnología                           | Justificación                                             | 5V asociada | Etapa             |
|----------------------------------------------|---------------------------------------------------|-----------------------------------------------------------|-------------|-------------------|
| Near real-time operativo (~500K eventos/día) | Spark Structured Streaming (Bronze stream)        | Latencia baja con micro-batches y control de late data    | Velocidad   | Parcial 2         |
| Batch de maestros/facturación (<100GB/día)   | Spark batch + Parquet particionado (Bronze batch) | Eficiente para datos periódicos y reproceso controlado    | Volumen     | Parcial 2         |
| Calidad de datos y trazabilidad              | Reglas + flags + quarantine en Silver             | Aisla datos inválidos sin frenar el pipeline              | Veracidad   | Parcial 2 y Final |
| Evolución de schema v1/v2                    | Compatibilización en Silver + contratos de schema | Reduce roturas por drift y habilita continuidad analítica | Variedad    | Parcial 2 y Final |
| Enriquecimiento y features de negocio        | Joins con org/users/resources + marts Gold        | Prepara 5 marts para FinOps, Soporte y Producto           | Valor       | Parcial 2 y Final |
| Idempotencia end-to-end                      | Checkpointing + keys naturales + upserts CQL      | Reejecución sin duplicados y consistencia en serving      | Veracidad   | Parcial 2 y Final |
| Serving para consultas de baja latencia      | AstraDB/Cassandra query-first (cloud_analytics)   | Modelo alineado a preguntas de negocio, sin full-scan     | Velocidad   | Parcial 2 y Final |
| Performance y escalabilidad                  | partitionBy + repartition/coalesce                | Control de costo/tiempo en Colab y escalabilidad futura   | Volumen     | Final             |

## 5Vs de Big Data aplicadas al proyecto

| V         | Aplicación concreta en Cloud Provider Analytics                                                                                       |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------|
| Volumen   | ~500K eventos/día via stream + <100GB/día en maestros y billing; almacenamiento en Parquet particionado por fecha                     |
| Velocidad | Ingesta streaming con micro-batches, watermark de 1h sobre event_ts y manejo de late data                                             |
| Variedad  | 7 fuentes CSV (batch) + JSONL multischema (stream): schema v1 (campos originales) y schema v2 (campos extendidos desde aprox. día 45) |
| Veracidad | Reglas de calidad por path, flags de validación por registro, quarantine de datos inválidos y trazabilidad de errores por capa        |
| Valor     | 5 marts Gold orientados a negocio: FinOps (uso, revenue, anomalías), Soporte (tickets) y Producto/GenAI (tokens)                      |

## 3) Flujo de datos (data pipeline)

Flujo operativo por path:

[PATH BATCH]

Landing (CSV maestros/billing)

- > PySpark DataFrame batch
- > Bronze Batch (Parquet, schema explícito, ingest\_ts)
- > Silver (validaciones, joins, quarantine)
- > Gold (marts FinOps, Soporte, Producto)
- > AstraDB/Cassandra (batch write via Spark connector)

[PATH STREAMING]

Landing (JSONL usage\_events\_stream)

- > PySpark Structured Streaming
- > Bronze Stream (Parquet, watermark 1h, dedupe event\_id, checkpoint)
- > Silver (validaciones stream, compatibilización schema v1/v2, quarantine)
- > Gold (actualización incremental de marts)
- > AstraDB/Cassandra (foreachBatch write)

[CONVERGENCIA]

Gold (ambos paths) --> AstraDB/Cassandra --> CQL --> BI/Dashboards

Herramientas por paso:

| Paso                         | Herramienta                               | Modo                 |
|------------------------------|-------------------------------------------|----------------------|
| Landing -> Bronze (maestros) | PySpark DataFrame                         | Batch diario/mensual |
| Landing -> Bronze (eventos)  | PySpark Structured Streaming              | Micro-batch continuo |
| Bronze -> Silver             | Spark SQL / DataFrame + reglas de calidad | Batch y trigger-once |
| Silver -> Gold               | Spark agregaciones por mart               | Batch y trigger-once |
| Gold -> Serving              | Spark-Cassandra connector / foreachBatch  | Batch y streaming    |
| Serving -> consumo           | CQL + BI (Super-set/Grafana/Tableau)      | On-demand            |

#### 4) Asunciones y riesgos iniciales

##### Asunciones

| #  | Asunción                                                               | Base técnica                                 | Mitigación si no se cumple                                                      |
|----|------------------------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------------------------|
| A1 | Volumen manejable en Google Colab (15GB RAM, sesiones de 12h)          | Datos de muestra + particionado desde Bronze | Particionado agresivo por fecha, coalesce, escalar a Colab Pro o GCS            |
| A2 | <code>event_id</code> es único y útil para deduplicación en stream     | Campo presente en ambas versiones de schema  | Reglas de unicidad compuesta + monitoreo de colisiones por batch                |
| A3 | Schema v2 comienza a aparecer aproximadamente en el día 45 del dataset | Evolución gradual documentada en los datos   | Contratos de schema explícitos + compatibilización en Silver desde día 1        |
| A4 | AstraDB free tier disponible (80GB, sin vencimiento de prueba)         | Plan gratuito documentado por DataStax       | Validación temprana de keyspace y fallback a Cassandra local via Docker         |
| A5 | Sin broker Kafka: streaming simulado desde archivos JSONL en disco     | Contexto académico en Colab                  | Arquitectura desacoplada que permite incorporar Kafka como fuente en producción |

##### Riesgos

| Riesgo                                            | Probabilidad | Impacto    | Mitigación                                                                              |
|---------------------------------------------------|--------------|------------|-----------------------------------------------------------------------------------------|
| OOM en Colab por crecimiento de datos             | Media        | Alta       | Procesamiento incremental, coalesce/repartition y ajuste de particiones por fecha       |
| Late data mayor al watermark de 1h                | Baja         | Media      | Ajustar watermark según observación empírica y auditar dropped events por batch         |
| Drift de schema no detectado a tiempo             | Media        | Alta       | Validaciones de schema explícito por capa + alertas en Silver si aparecen campos nuevos |
| Duplicados en serving por falla de idempotencia   | Media        | Alta       | Checkpointing estricto + upserts por clave natural en Cassandra (INSERT IF NOT EXISTS)  |
| Hot partitions en Cassandra por baja cardinalidad | Media        | Media/Alta | Revisar cardinalidad de partition key y agregar bucketing si es necesario               |
| Acople alto entre notebooks y lógica de negocio   | Media        | Media      | Modularizar transformaciones en funciones reutilizables por capa                        |

## 5) Estimación de esfuerzo y recursos (rough estimate)

Supuesto de equipo: 3 personas.

| Bloque                               | Tiempo estimado | Roles principales              |
|--------------------------------------|-----------------|--------------------------------|
| Diseño y base técnica (esta entrega) | 3 a 4 días      | Data Architect + Data Engineer |

| Bloque                                                | Tiempo estimado | Roles principales                                   |
|-------------------------------------------------------|-----------------|-----------------------------------------------------|
| Ingesta Bronze<br>batch/stream +<br>idempotencia      | 4 a 6 días      | Data Engineer Batch +<br>Data Engineer<br>Streaming |
| Silver (calidad +<br>quarantine +<br>enriquecimiento) | 4 a 6 días      | Data Engineer +<br>Analytics Engineer               |
| Gold mínimo + modelo<br>Cassandra (Parcial 2)         | 3 a 5 días      | Data Engineer + Data<br>Modeler                     |
| Expansión de marts +<br>optimización (Final)          | 6 a 9 días      | Data Engineer +<br>Analytics Engineer               |
| Storytelling,<br>presentación y demo<br>final         | 3 a 4 días      | Todo el equipo                                      |

**Total estimado:** 23 a 34 días-persona distribuidos en ~3 semanas de trabajo en equipo.

#### Recursos y costos

| Recurso                                | Tier utilizado                          | Costo estimado        |
|----------------------------------------|-----------------------------------------|-----------------------|
| Google Colab                           | Free (15GB RAM, GPU básica, 12h sesión) | USD 0                 |
| Google Drive                           | Free (15GB almacenamiento)              | USD 0                 |
| AstraDB (Cassandra gestionado)         | Free (80GB, sin vencimiento)            | USD 0                 |
| Herramienta BI (Apache Superset)       | Open source, self-hosted en Colab       | USD 0                 |
| <b>Escenario base</b>                  | <b>Todo free tier</b>                   | <b>USD 0/mes</b>      |
| Colab Pro (si se requiere más RAM/GPU) | Pro plan                                | USD 10,49/mes         |
| <b>Escenario extendido</b>             | <b>Colab Pro</b>                        | <b>~USD 10,49/mes</b> |

## 6) Decisiones para facilitar entregas futuras

- Estandarizar nomenclatura de datasets y particiones desde Bronze (/bronze/batch/, /bronze/stream/, /silver/, /gold/, /quarantine/, /checkpoints/).
- Mantener contratos de schema por capa (StructType declarado en código, no inferido).
- Diseñar Gold por preguntas de negocio (query-first) para que el modelo Cassandra sea directo.

- Construir trazabilidad de calidad desde el MVP para evitar retrabajo en la entrega final.
- Documentar decisiones y trade-offs por iteración para reutilizar en presentación y video final.

## Evidencia de ejecucion (Segundo Parcial)

## Evidencia de ejecucion - Segundo Parcial (MVP tecnico)

### Conteos por capa

| Dataset                         | Filas | Path                                         |
|---------------------------------|-------|----------------------------------------------|
| bronze_batch/customers_orgs     | 3     | /home/pipemind/big/datalake/bronze/batch/cus |
| bronze_batch/users              | 4     | /home/pipemind/big/datalake/bronze/batch/use |
| bronze_batch/billing_monthly    | 3     | /home/pipemind/big/datalake/bronze/batch/bil |
| bronze_stream/usage_events      | 5     | /home/pipemind/big/datalake/bronze/stream/us |
| quarantine/late_data            | 1     | /home/pipemind/big/datalake/quarantine/late_ |
| silver/usage_enriched           | 3     | /home/pipemind/big/datalake/silver/usage_enr |
| silver/daily_features           | 3     | /home/pipemind/big/datalake/silver/daily_fea |
| quarantine/silver_quality       | 2     | /home/pipemind/big/datalake/quarantine/silve |
| gold/org_daily_usage_by_service | 3     | /home/pipemind/big/datalake/gold/org_daily_u |

### Reglas de calidad y quarantine (muestra)

| event_id | quality_issue        | service | org_id  |
|----------|----------------------|---------|---------|
| e_003    | cost_lt_-0.01        | storage | org_002 |
| e_006    | unit_null_with_value | compute | org_003 |

### Muestra Gold (FinOps)

| org_id  | service  | usage_date | daily_cost_usd | requests | genai_tokens | carbon_kg | has_cost_anomaly |
|---------|----------|------------|----------------|----------|--------------|-----------|------------------|
| org_001 | compute  | 2026-05-16 | 8.4            | 120.0    | 0.0          | 1.6       | False            |
| org_001 | genai    | 2026-05-16 | 4.1            | 25.0     | 5000.0       | 0.3       | False            |
| org_003 | database | 2026-05-16 | 1.8            | 200.0    | 0.0          | 0.0       | False            |



## Validacion de idempotencia

Este reporte debe generarse luego de ejecutar dos veces `python scripts/run_mvp.py`. Si los conteos de Gold permanecen estables, la re-ejecucion no esta duplicando filas.

---

## Evidencia de idempotencia

### Evidencia de idempotencia

Se ejecuto `python scripts/run_mvp.py` dos veces consecutivas sobre datos sintéticos generados por `scripts/bootstrap_sample_landing.py`.

### Conteos actuales por dataset

| dataset            | filas |
|--------------------|-------|
| bronze_customers   | 3     |
| bronze_users       | 4     |
| bronze_billing     | 3     |
| bronze_stream      | 5     |
| silver_usage       | 3     |
| silver_features    | 3     |
| quarantine_quality | 2     |
| quarantine_late    | 1     |
| gold_finops        | 3     |

## Conclusión

- Los conteos son consistentes con la ejecucion esperada en el entorno local.
  - La re-ejecucion no incrementó filas en Gold.
- 

## MVP checklist y bitacora

### Segundo Parcial - Checklist de evidencia (MVP tecnico)

#### Objetivo

Demostrar flujo end-to-end minimo: Landing -> Bronze -> Silver -> Gold -> Serving (Cassandra)

## Evidencias que deben adjuntar

- ☐ Bronze batch generado para 3 maestros:
  - customers\_orgs
  - users
  - billing\_monthly
- ☐ Bronze streaming generado desde `usage_events_stream/*.jsonl`.
- ☐ Watermark + dedupe por `event_id` + checkpoint activo.
- ☐ Silver con 3 reglas de calidad aplicadas.
- ☐ Quarantine poblada con ejemplos invalidos.
- ☐ Gold con `org_daily_usage_by_service` generado.
- ☐ Cassandra: keyspace y tabla(s) creadas con `cassandra/schema.cql`.
- ☐ Dos consultas minimas ejecutadas (`cassandra/queries_minimas.cql`).
- ☐ Evidencia de idempotencia (re-ejecucion sin duplicados).

## Comandos recomendados

```
python -m pip install -r requirements.txt
python scripts/run_mvp.py
```

Con carga a Cassandra:

```
python scripts/run_mvp.py --with-cassandra --cassandra-host 127.0.0.1 --cassandra-port 9042
```

## Evidencia de idempotencia sugerida

1. Ejecutar `python scripts/run_mvp.py` dos veces.
2. Comparar conteos en Gold antes y despues:

```
spark.read.parquet("datalake/gold/org_daily_usage_by_service").count()
```

3. Verificar que no crecen filas por duplicado al reprocesar mismos archivos.

## Bitacora de apropiacion tecnica

Este archivo registra decisiones tomadas por el equipo para demostrar apropiacion del trabajo tecnico.

## Decisiones clave tomadas por el equipo

1. Se eligio Lambda y no Kappa porque el dominio combina fuentes naturalmente batch y streaming.
2. En streaming se uso watermark de 1 hora para balancear precision y complejidad operativa.
3. Se implemento quarantine en Silver para no bloquear el pipeline por errores de calidad.
4. Se modelo Cassandra por consulta (query-first), incluyendo tabla auxiliar para Top-N 14 dias.

5. Se priorizo idempotencia en todas las capas para permitir reproceso seguro.

### **Trade-offs explicitados**

- Simplicidad en Colab vs escalabilidad total de produccion.
- Tabla auxiliar en Cassandra para query #2 vs agregacion ad-hoc por cliente.
- `availableNow` para reproducibilidad del MVP vs stream continuo de larga vida.

### **Preguntas de defensa sugeridas (oral)**

- Por que watermark de 1h y no 10m o 6h.
- Como cambiaria el diseno al pasar de JSONL files a Kafka.
- Que impacto tienen las claves de particion elegidas en Cassandra.
- Que reglas de calidad se endurecerian primero para produccion.